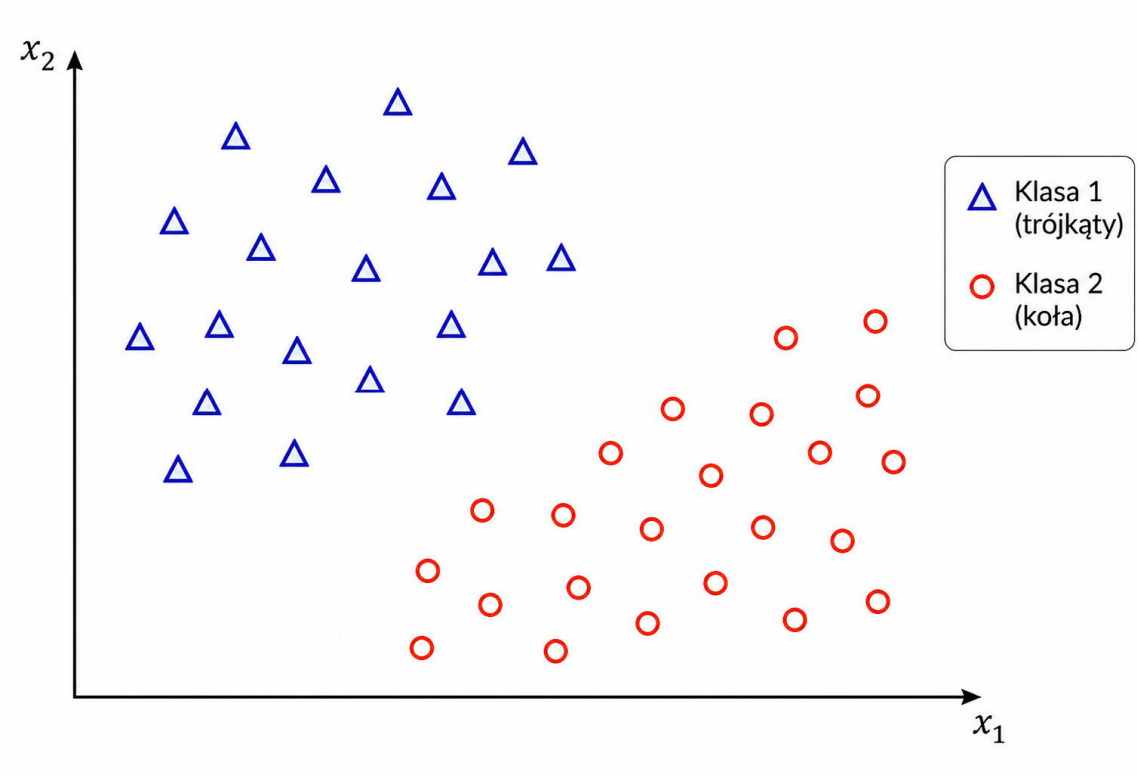


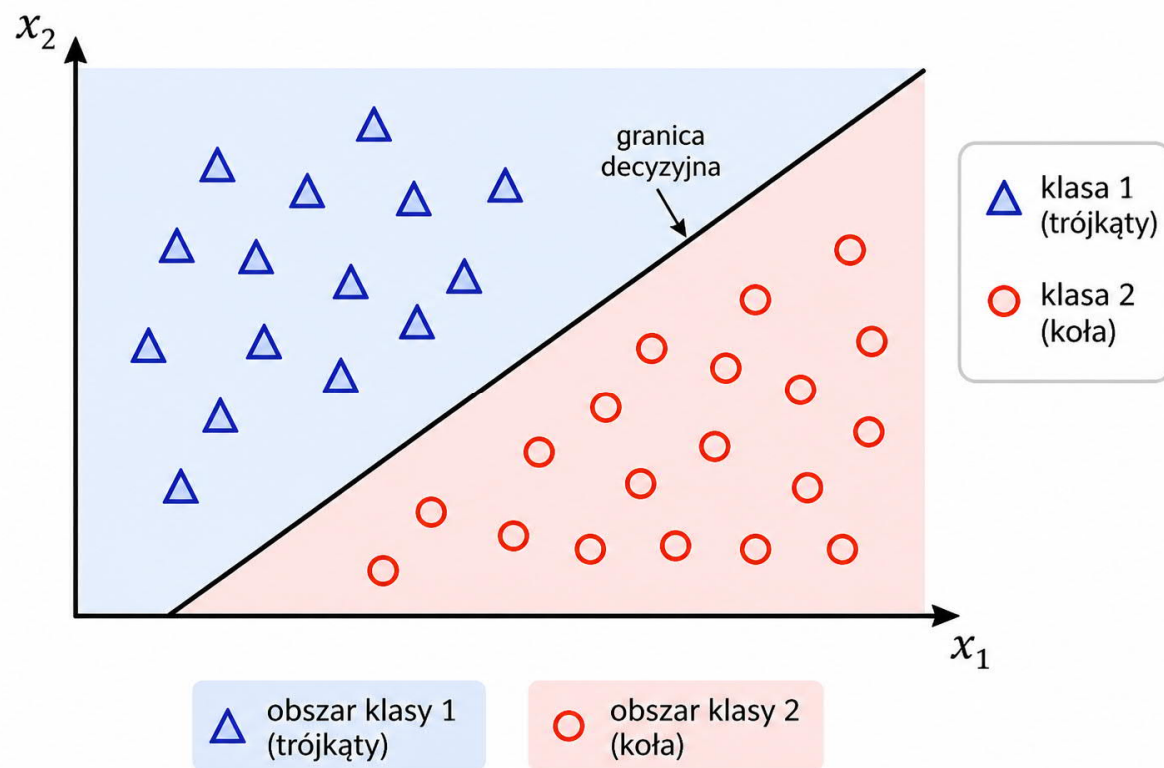


PERCEPTRON

WOJCIECH DOBROSIELSKI



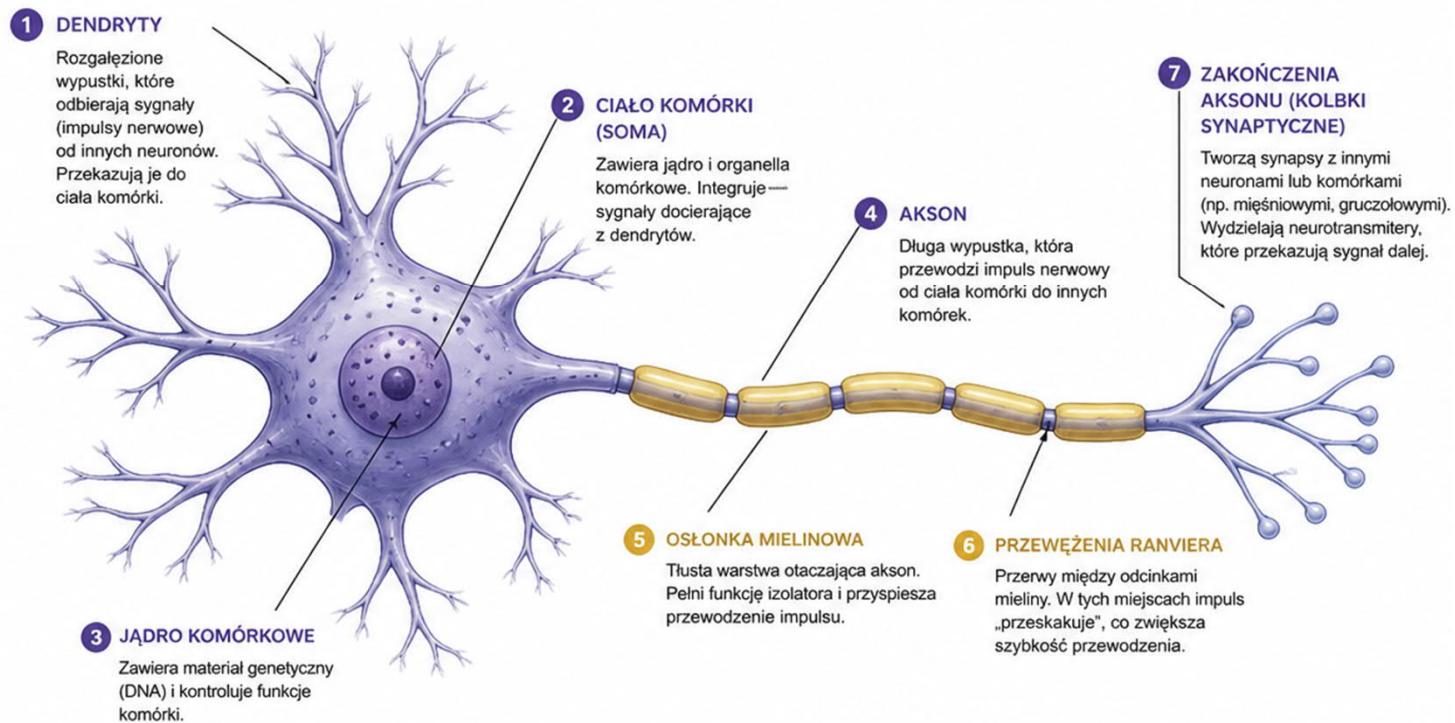
JAK PODZIELIĆ, OKREŚLIĆ GRANICĘ OBIEKTÓW?



JAK PODZIELIĆ, OKREŚLIĆ GRANICĘ OBIEKTÓW?

ŁUDZKI NEURON

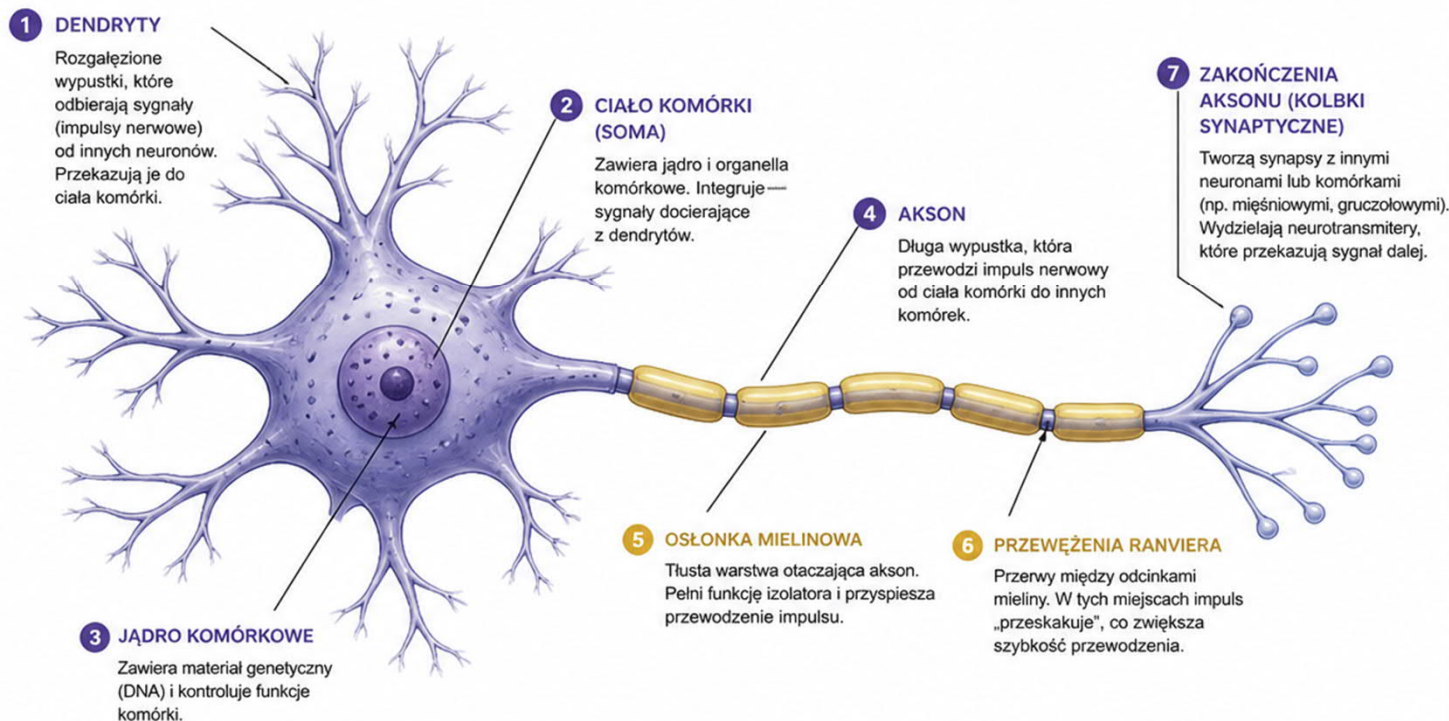
Podstawowa jednostka układu nerwowego odpowiedzialna za przetwarzanie i przekazywanie informacji.



NEURON

LUDZKI NEURON

Podstawowa jednostka układu nerwowego odpowiedzialna za przetwarzanie i przekazywanie informacji.



JAK DZIAŁA NEURON?

- 1** Dendryty odbierają sygnały od innych komórek.
- 2** Sygnały są integrowane w ciele komórki.
- 3** Jeśli suma sygnałów osiągnie próg pobudzenia, w aksonie powstaje impuls nerwowy (potencjał czynnościowy).
- 4** Impuls przewodzony jest wzdłuż aksonu.
- 5** Na zakończeniach aksonu impuls powoduje uwolnienie neuroprzekaźników.
- 6** Neuroprzekaźniki przekazują sygnał następnej komórce przez synapsę.

NEURON

LUDZKI NEURON

Podstawowa jednostka układu nerwowego odpowiedzialna za przetwarzanie i przekazywanie informacji.

JAK DZIAŁA NEURON?

- 1 Dendryty odbierają sygnały od innych komórek.



- 2 Sygnały są integrowane w ciele



- 5 Na zakończeniach aksonu impuls powoduje uwolnienie neuroprzekaźników.

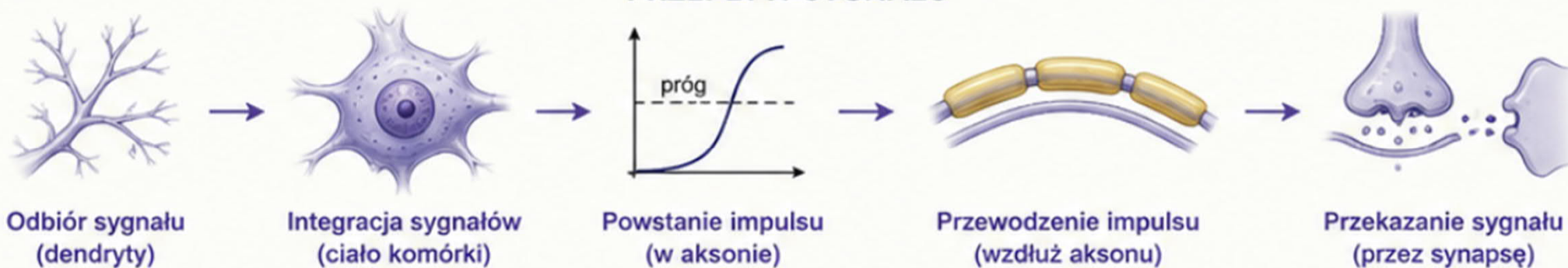


- 6 Neuroprzekaźniki przekazują sygnał następnej komórce przez synapsę.

- 1 DENDRYTY
Rozgałęzione wypustki, które

- 7 ZAKOŃCZENIA

PRZEPŁYW SYGNAŁU

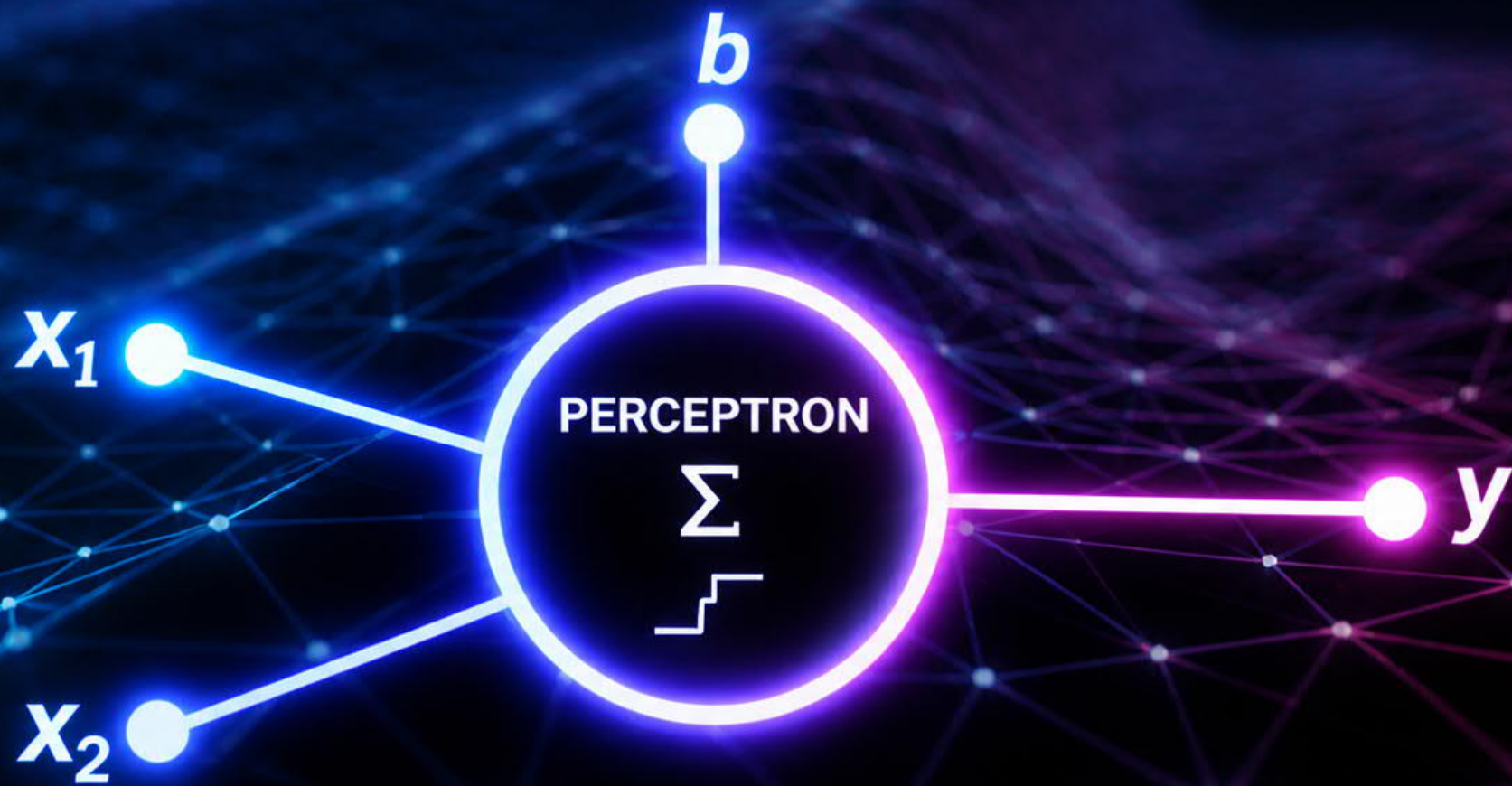


- 3 JĄDRO KOMÓRKOWE
Zawiera materiał genetyczny (DNA) i kontroluje funkcje komórki.

7. Warstwa izolacyjna aksonu.
Pełni funkcję izolatora i przyspiesza przewodzenie impulsu.

Przerwy między odcinkami mieliny. W tych miejscach impuls „przeskakuje”, co zwiększa szybkość przewodzenia.

NEURON



Perceptron można traktować jako bardzo uproszczony model neuronu opisany matematycznie.

Początkowo rozwój sztucznych sieci neuronowych był inspirowany próbą naśladowania sposobu, w jaki ludzki mózg uczy się i uogólnia informacje. Z czasem jednak podejście to stało się bardziej praktyczne — dziś sieci neuronowe wykorzystuje się głównie do rozwiązywania konkretnych problemów obliczeniowych.

Pojedynczy perceptron jest w stanie realizować proste zadania, takie jak klasyfikacja lub regresja.

Jednocześnie stanowi on fundament dla bardziej zaawansowanych struktur — rozbudowanych sieci neuronowych, które radzą sobie ze znacznie bardziej złożonymi problemami.

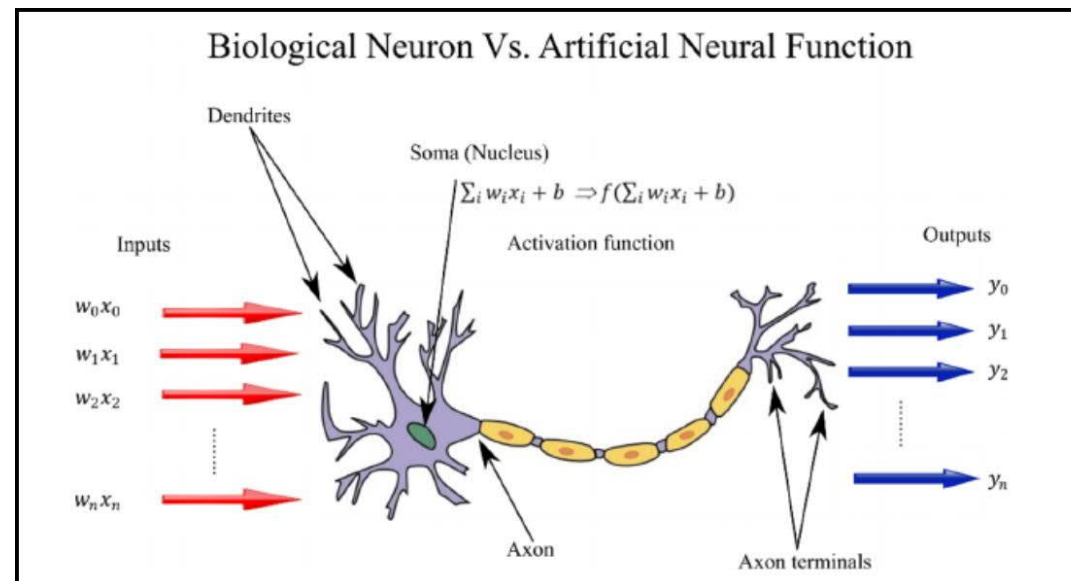
PERCEPTRON

Twórcą perceptronu jest Frank Rosenblatt.

Opracował go w 1957 roku, pracując nad modelami uczenia maszynowego inspirowanymi działaniem mózgu.

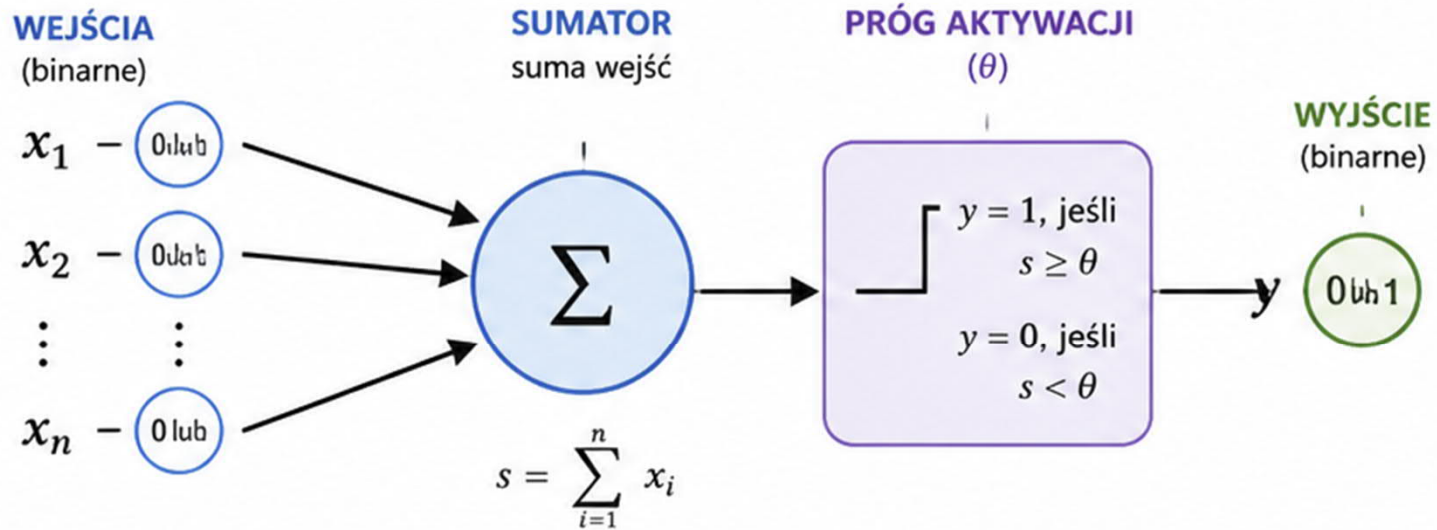
Jego celem było stworzenie systemu, który potrafi uczyć się na podstawie danych — podobnie jak człowiek.

Perceptron był jednym z pierwszych kroków w rozwoju sztucznych sieci neuronowych i stanowi fundament dla dzisiejszych metod uczenia maszynowego.



MODEL ROSENBLATTA

1. STRUKTURA NEURONU



WZÓR OGÓLNY

$$y = \begin{cases} 1, & \text{jeśli } \sum_{i=1}^n x_i \geq \theta \\ 0, & \text{jeśli } \sum_{i=1}^n x_i < \theta \end{cases}$$

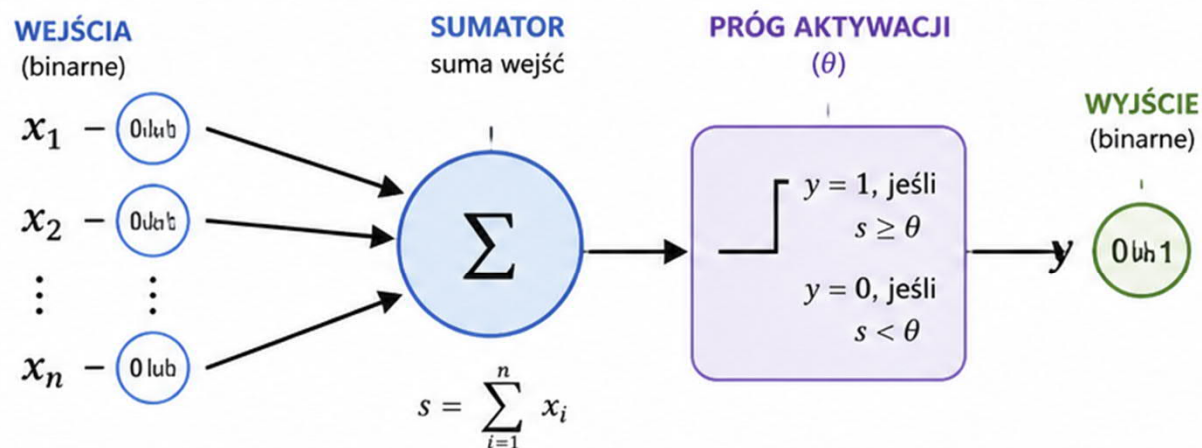
x_i – wejścia (0 lub 1)

θ – próg aktywacji (liczba naturalna)

y – wyjście (0 lub 1)

ROZWIĄZANIE MCCULLOCHA I PITTSA (1943)

1. STRUKTURA NEURONU



WZÓR OGÓLNY

$$y = \begin{cases} 1, & \text{jeśli } \sum_{i=1}^n x_i \geq \theta \\ 0, & \text{jeśli } \sum_{i=1}^n x_i < \theta \end{cases}$$

x_i – wejścia (0 lub 1)
 θ – próg aktywacji (liczba naturalna)
 y – wyjście (0 lub 1)

2. JAK DZIAŁA?

- 1 Neuron otrzymuje sygnały wejściowe $x_i \in \{0, 1\}$.
- 2 Oblicza sumę wszystkich wejść:
$$s = \sum_{i=1}^n x_i$$
- 3 Porównuje sumę s z progiem θ :
 - jeśli $s \geq \theta \rightarrow y = 1$ (neuron aktywny)
 - jeśli $s < \theta \rightarrow y = 0$ (neuron nieaktywny)

3. KLUCZOWE CECHY

- ✓ Wejścia binarne (0 lub 1)
- ✓ Brak wag (wszystkie wagi = 1)
- ✓ Stały próg aktywacji θ
- ✓ Brak mechanizmu uczenia się
- ✓ Działa jak bramka logiczna (np. AND, OR, NOT)

ROZWIĄZANIE MCCULLOCHA I PITTSA (1943)

1. STRUKTURA NEURONU

WEJŚCIA
(binarne)

SUMATOR
suma wejść

PRÓG AKTYWACJI
(θ)

2. JAK DZIAŁA?

1 Neuron otrzymuje sygnały wejściowe $x_i \in \{0, 1\}$.

2 Oblicza sumę wszystkich wejść:

$$s = \sum_{i=1}^n x_i$$

4. PRZYKŁADY DZIAŁANIA – BRAMKI LOGICZNE

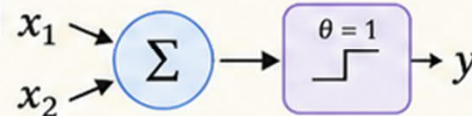
BRAMKA AND



Neuron aktywny
tylko gdy oba
wejścia = 1

x_1	x_2	$s = x_1 + x_2$	$y (\theta = 2)$
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

BRAMKA OR



Neuron aktywny,
gdy przynajmniej jedno
wejście = 1

x_1	x_2	$s = x_1 + x_2$	$y (\theta = 1)$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	2	1

$$y = \begin{cases} 0, & \text{jeśli } \sum_{i=1}^n x_i < \theta \end{cases}$$

θ – próg aktywacji (liczba naturalna)
 y – wyjście (0 lub 1)

- ✓ Stały próg aktywacji θ
- ✓ Brak mechanizmu uczenia się
- ✓ Działa jak bramka logiczna (np. AND, OR, NOT)

ROZWIĄZANIE MCCULLOCHA I PITTSA (1943)

Model McCullocha–Pittsa (1943)

- wejścia: tylko **0 lub 1**,
- brak wag (lub stałe),
- działa jak **bramka logiczna (AND, OR, NOT)**,
- ma próg aktywacji,
- **nie uczy się**,
- zastosowanie: logika, teoria obliczeń

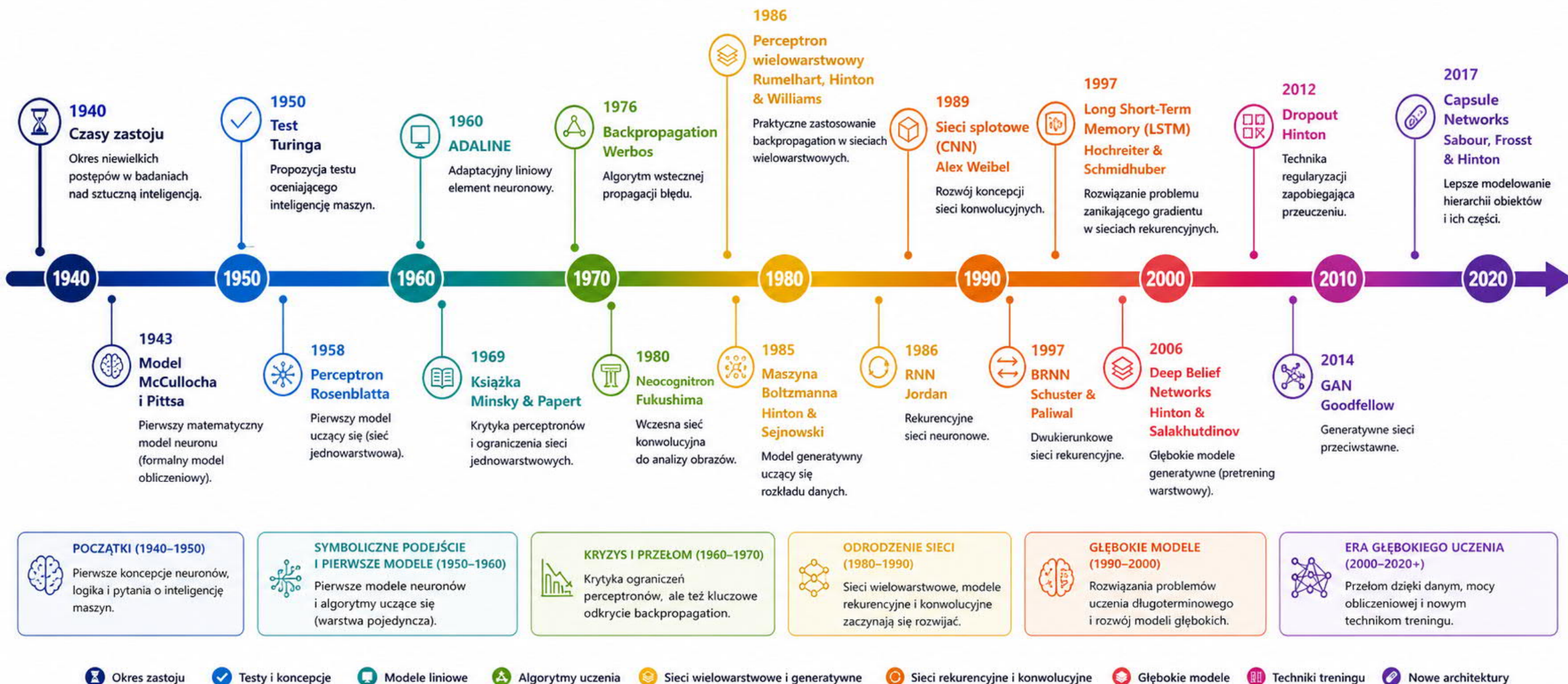
Perceptron Rosenblatta (1957)

- wejścia: liczby rzeczywiste,
- posiada wagi ($w_1, w_2, \dots$) i bias,
- decyzja na podstawie sumy ważonej,
- uczy się (modyfikuje wagi na podstawie błędu),
- zastosowanie: klasyfikacja danych

Perceptron to rozwinięcie modelu McCullocha–Pittsa o wagi i mechanizm uczenia.

RÓŻNICE

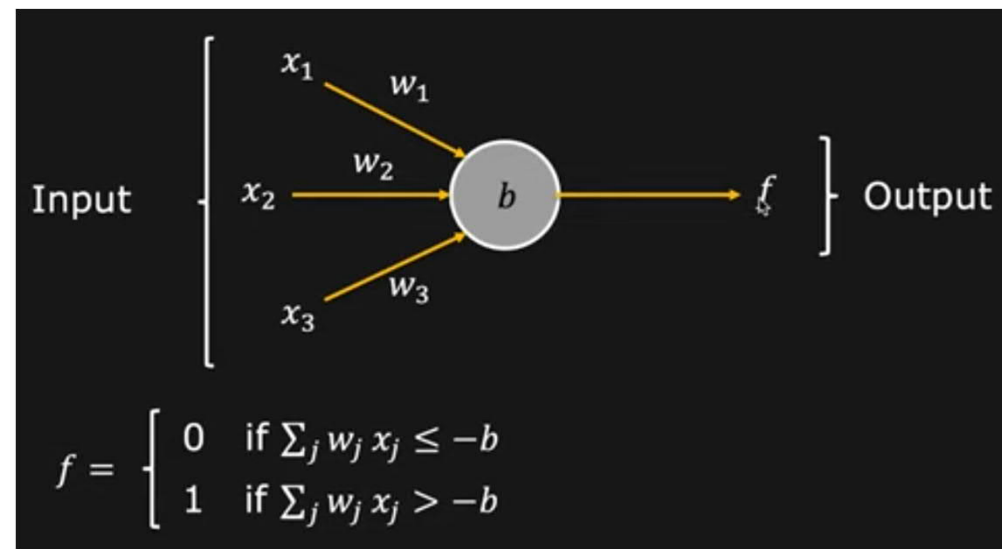
OŚ CZASU – ROZWÓJ SIECI NEURONOWYCH I GŁĘBOKIEGO UCZENIA



Po lewej stronie znajdują się trzy wejścia x_1 , x_2 i x_3 , które reprezentują dane wejściowe. Każde z tych wejść jest połączone z neuronem za pomocą wag w_1 , w_2 i w_3 , określających znaczenie poszczególnych sygnałów.

W centralnej części znajduje się neuron oznaczony jako b , który pełni rolę biasu. Neuron ten oblicza sumę ważoną wszystkich wejść, czyli mnoży każde wejście przez odpowiednią wagę i dodaje do tego wartość biasu.

Po wykonaniu obliczeń sygnał jest przekazywany na wyjście, oznaczone jako f . Wyjście to stanowi wynik działania perceptronu, czyli jego decyzję lub odpowiedź na podstawie dostarczonych danych wejściowych.



MODEL ROSENBLATTA

Funkcja aktywacji ma postać:

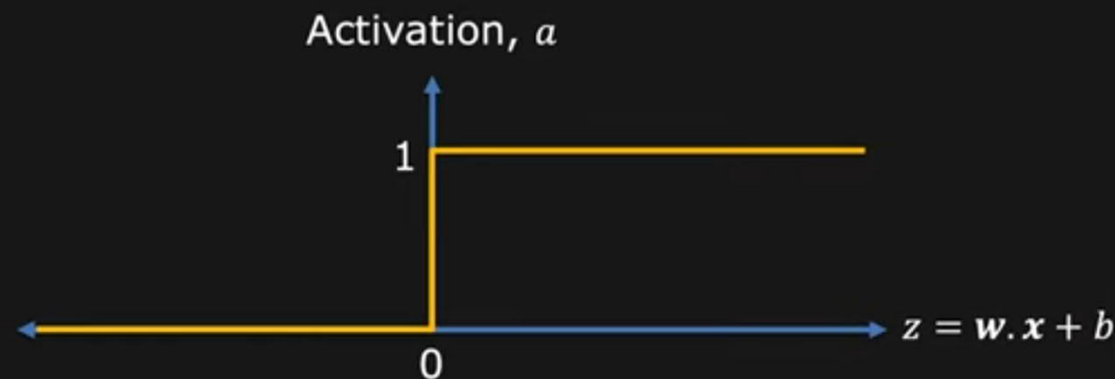
- $a = 0$, gdy $z \leq 0$
- $a = 1$, gdy $z > 0$

Wykres tej funkcji:

- oś pozioma przedstawia wartość $z = w \cdot x + b$
- oś pionowa przedstawia wyjście a
- dla wartości $z \leq 0$ funkcja ma wartość 0
- w punkcie $z = 0$ następuje skok
- dla $z > 0$ funkcja przyjmuje wartość 1

Let: $z = w \cdot x + b$

$$\text{Activation: } a = f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$



FUNKCJĘ AKTYWACJI PERCEPTRONU (FUNKCJĘ PROGOWĄ)

Czy zamówić jedzenie na dowóz?

Głód

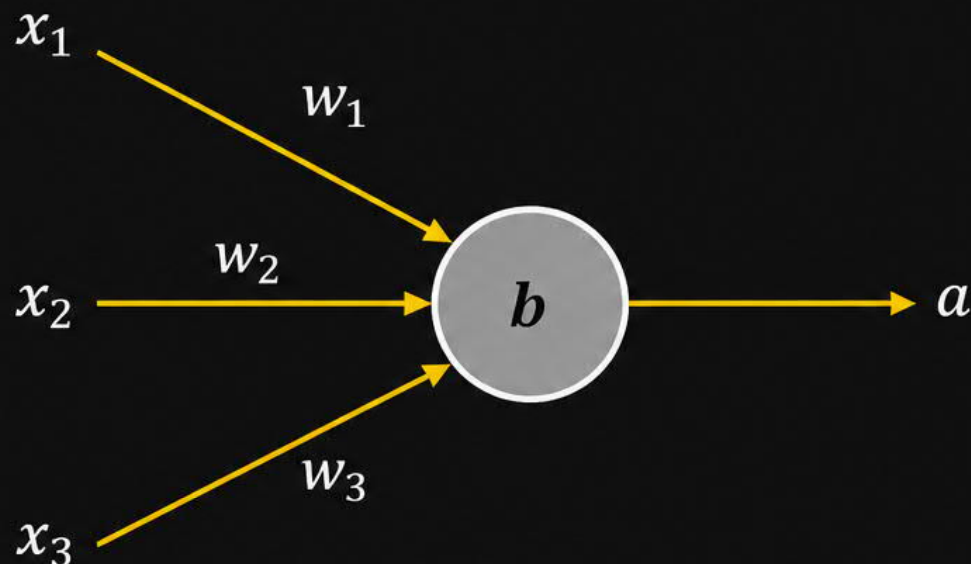
x_1 : Czy jesteś głodny?

Pora dnia

x_2 : Czy jest późno
(np. wieczór)?

Budżet

x_3 : Czy masz pieniądze
na zamówienie?



Aktywacja (decyzja)

$a = 1 \rightarrow$ **zamawiasz** 🍕

$a = 0 \rightarrow$ **nie zamawiasz** ✗

x_1 – głód

x_2 – pora dnia

x_3 – budżet

Model analizuje wszystkie czynniki jednocześnie, przypisuje im różne wagi (w_1, w_2, w_3), dodaje **bias** b i podejmuje decyzję.

Przykładowe wagi (znaczenie czynników):

- w_1 (głód) = **2.0** – bardzo ważny
- w_2 (pora dnia) = **1.0** – średnio ważny
- w_3 (budżet) = **-1.5** – działa przeciwnie



INTUICJA:

Duży głód i późna pora zwiększają szansę zamówienia.
Brak pieniędzy zmniejsza szansę.

Interpretację geometryczną modelu.

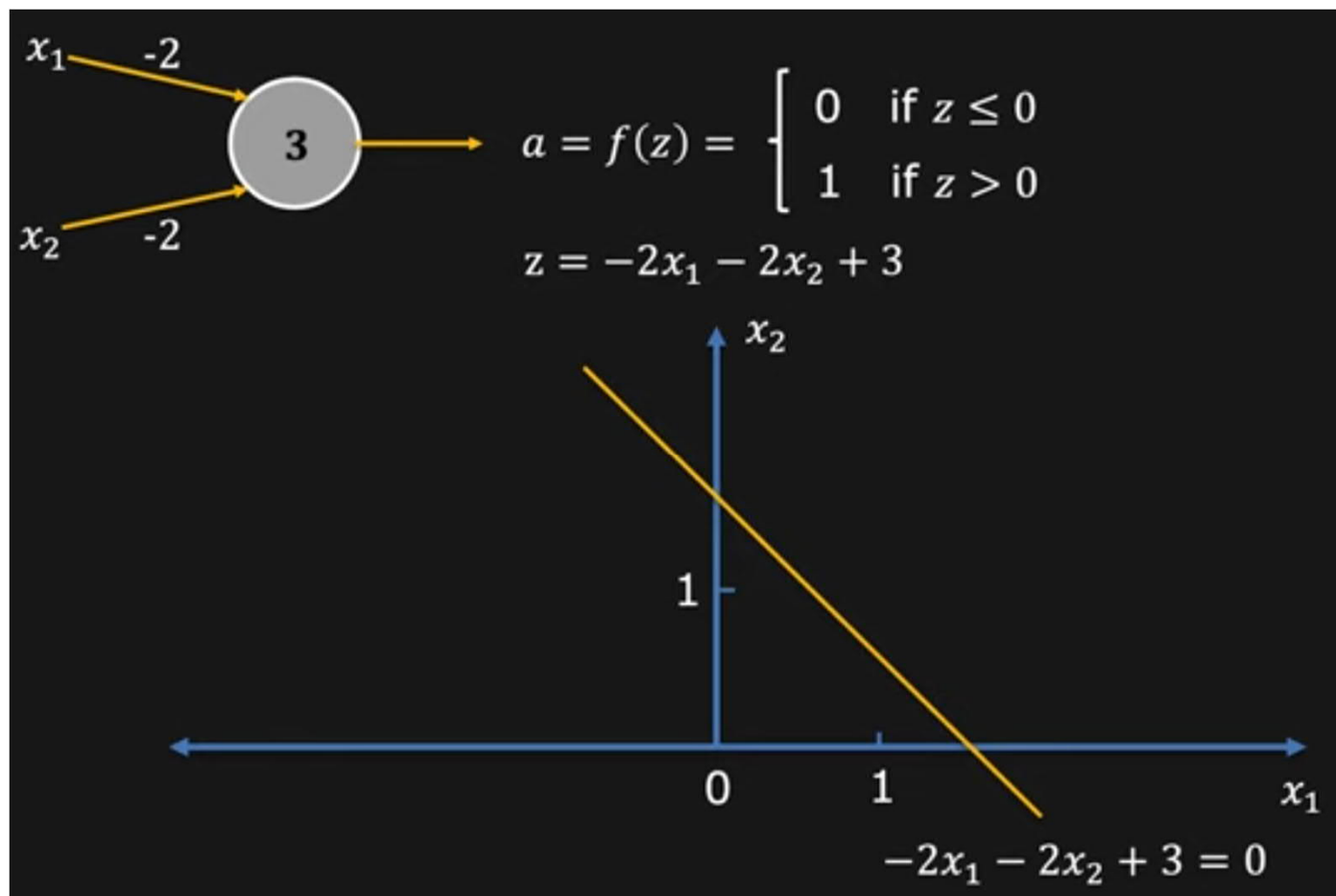
Prosta opisana równaniem:

$$-2x_1 - 2x_2 + 3 = 0$$

stanowi granicę decyzyjną, która dzieli przestrzeń na dwa obszary:

- po jednej stronie perceptron zwraca 1
- po drugiej stronie zwraca 0

Oznacza to, że perceptron klasyfikuje dane na podstawie ich położenia względem tej prostej.



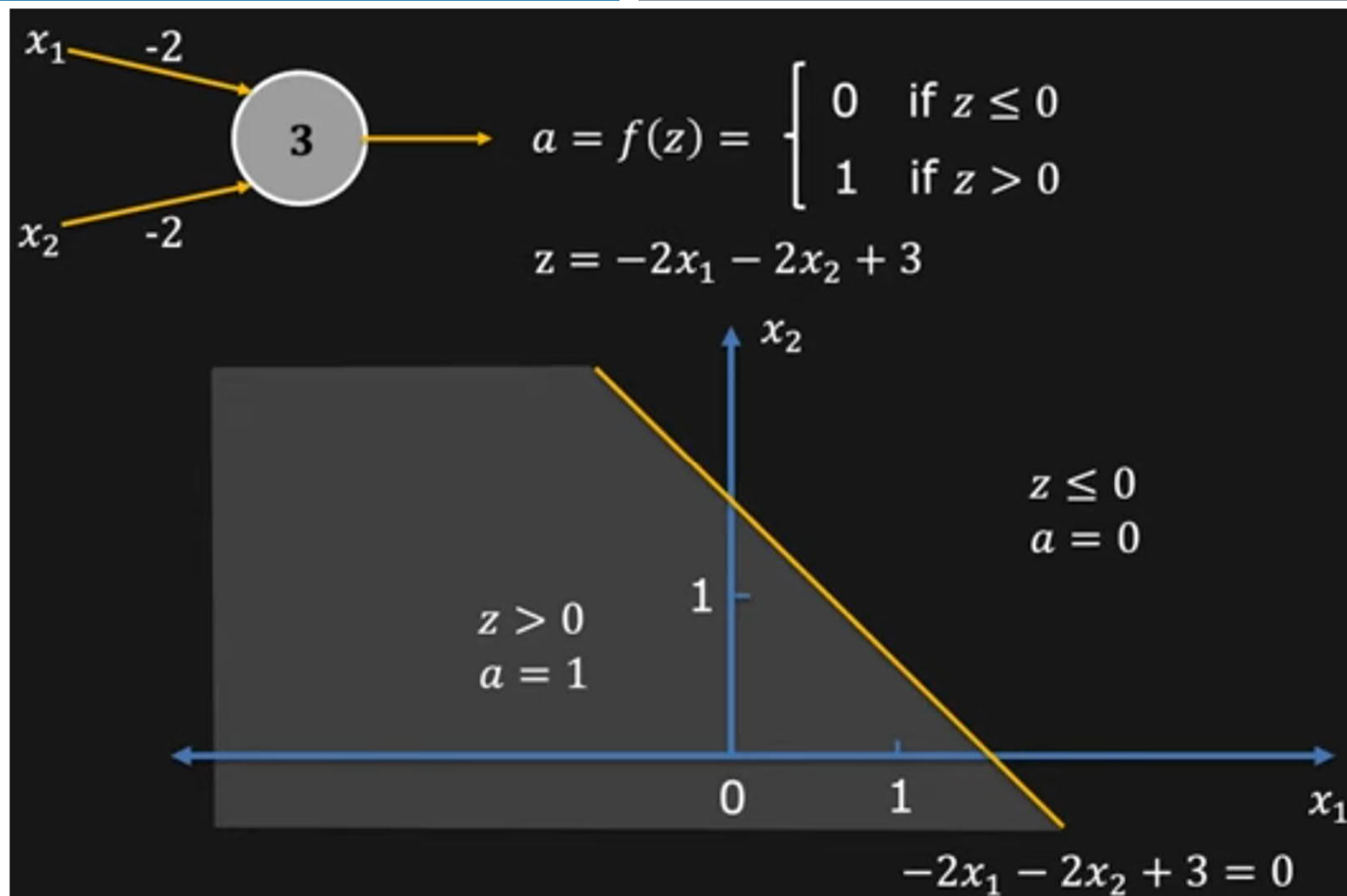
PERCEPTRON JAKO LINIOWA KLASYFIKACJA

Prosta opisana równaniem:

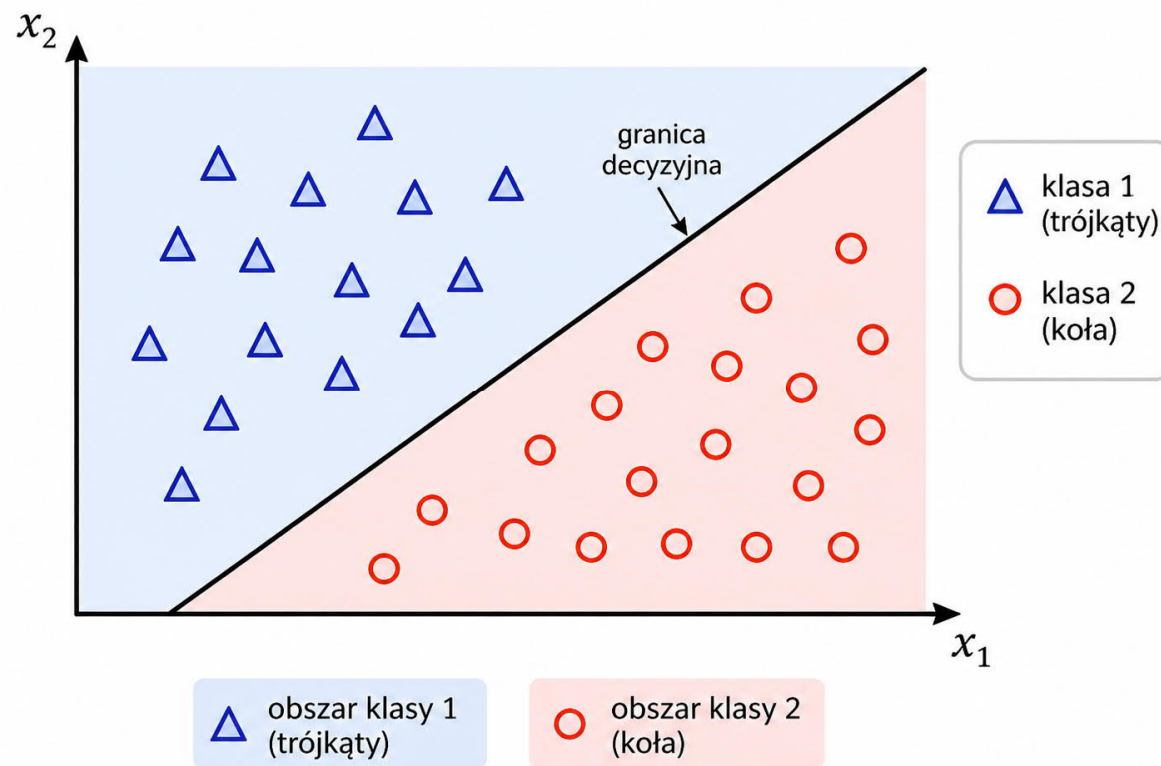
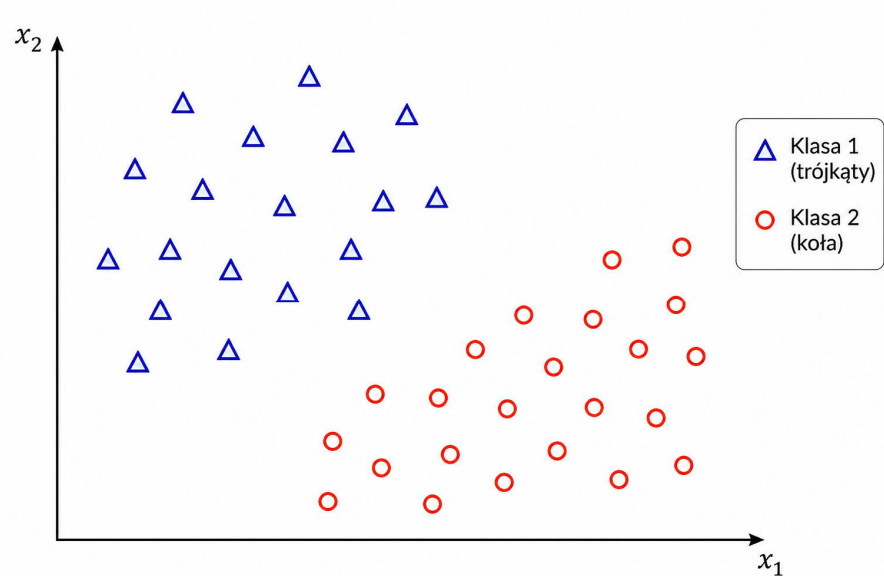
$$-2x_1 - 2x_2 + 3 = 0$$

stanowi granicę decyzyjną, która dzieli przestrzeń na dwa obszary.

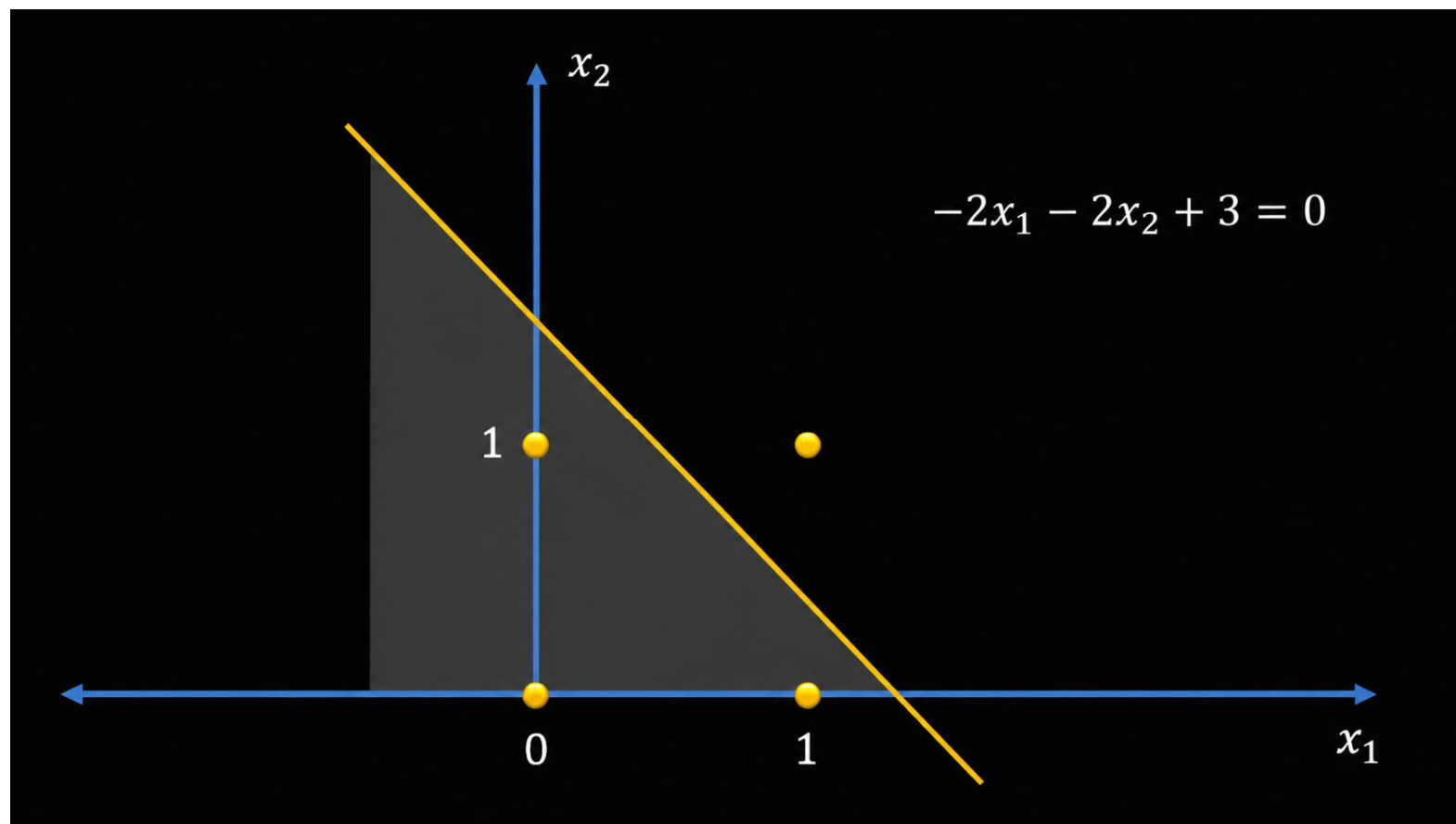
- Obszar zacieniowany odpowiada przypadkom, dla których $z > 0$, więc perceptron zwraca $a = 1$,
- Obszar poza nim odpowiada sytuacjom, gdzie $z \leq 0$, czyli wynik to $a = 0$.



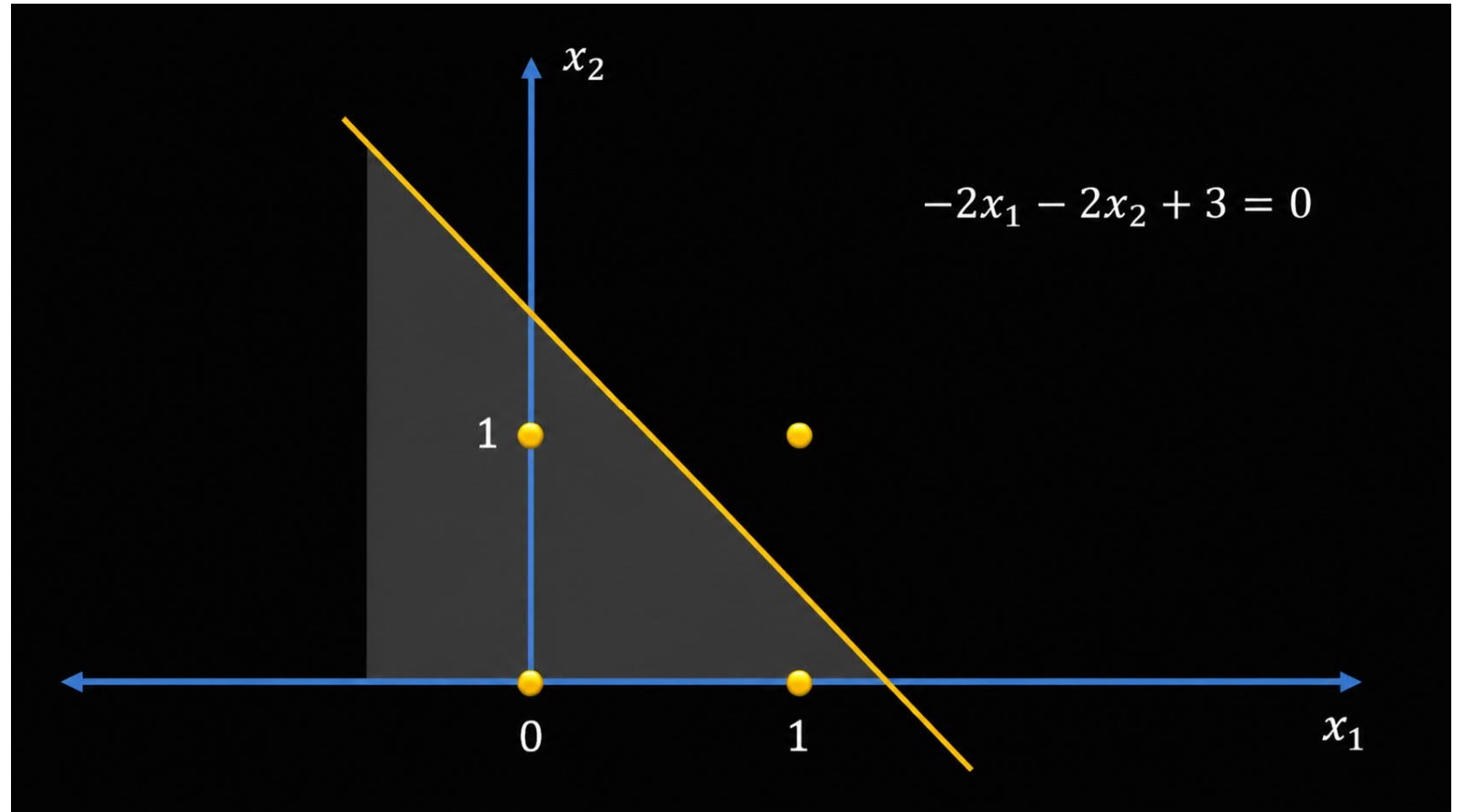
PERCEPTRON JAKO LINIOWA KLASYFIKACJA



JAK PODZIELIĆ OBIEKTY, UŻYJ PERCEPTRONU



JAKI PROBLEM PRZEDSTAWIA RYSUNEK?



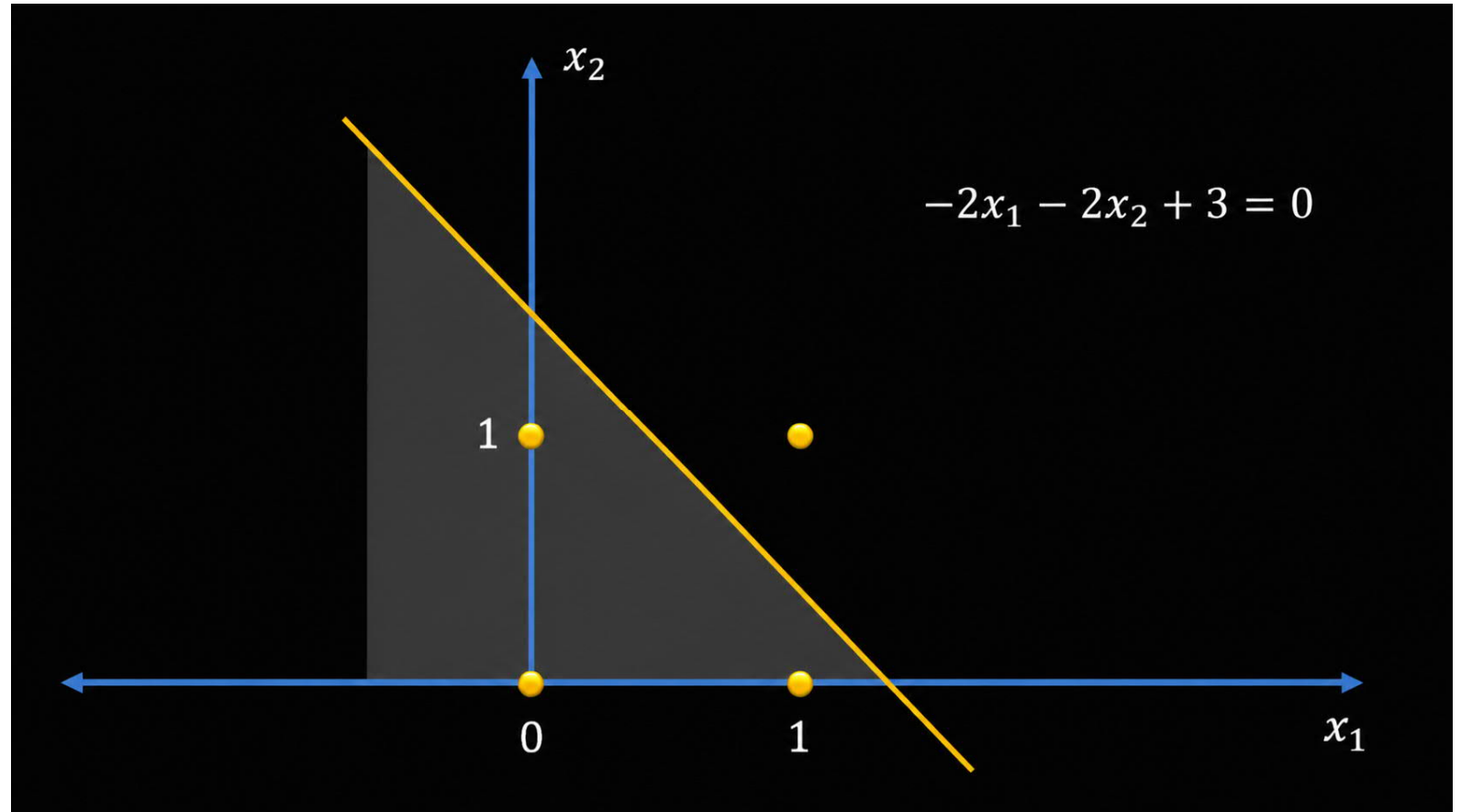
JAKA TO JEST BRAMKA LOGICZNA?

$$\text{NAND}(x_1, x_2) = \neg(x_1 \wedge x_2)$$

BRAMKA NAND

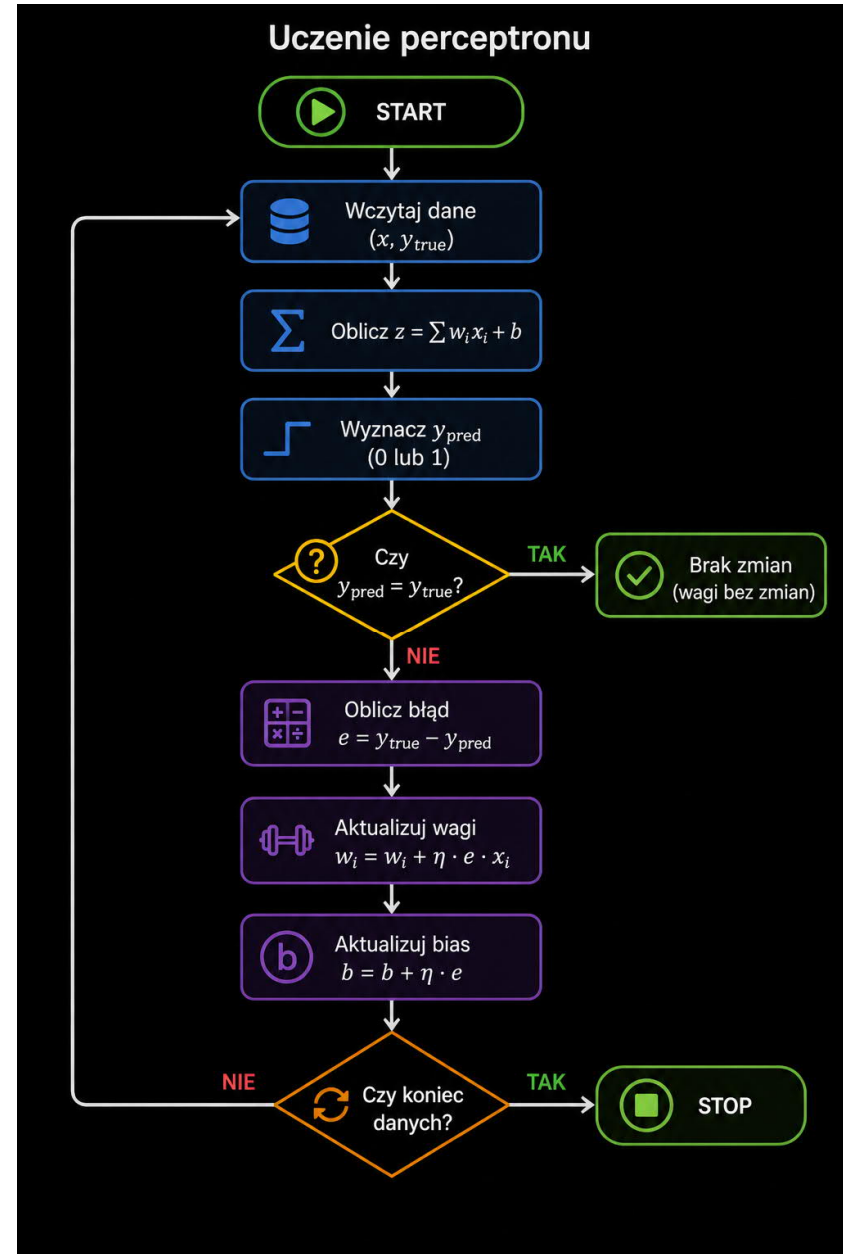
x_1	x_2	NAND
0	0	1
0	1	1
1	0	1
1	1	0

BRAMKA LOGICZNA NAND



Uczenie w perceptronie polega na dostosowywaniu wag i biasu na podstawie popełnianych błędów, tak aby model poprawnie klasyfikował dane.

JAK WYGLĄDA UCZENIE W PERCEPTRONIE



Obliczenie wyniku

Najpierw liczymy:

$$z = \sum w_i x_i + b$$

Następnie:

- jeśli $z > 0 \rightarrow y_{pred} = 1$,
- jeśli $z \leq 0 \rightarrow y_{pred} = 0$.

Porównanie z prawdą

- y_{true} –poprawna odpowiedź,
- y_{pred} –przewidywanie modelu.

liczymy błąd:

$$\text{błąd} = y_{true} - y_{pred}$$

Aktualizacja wag

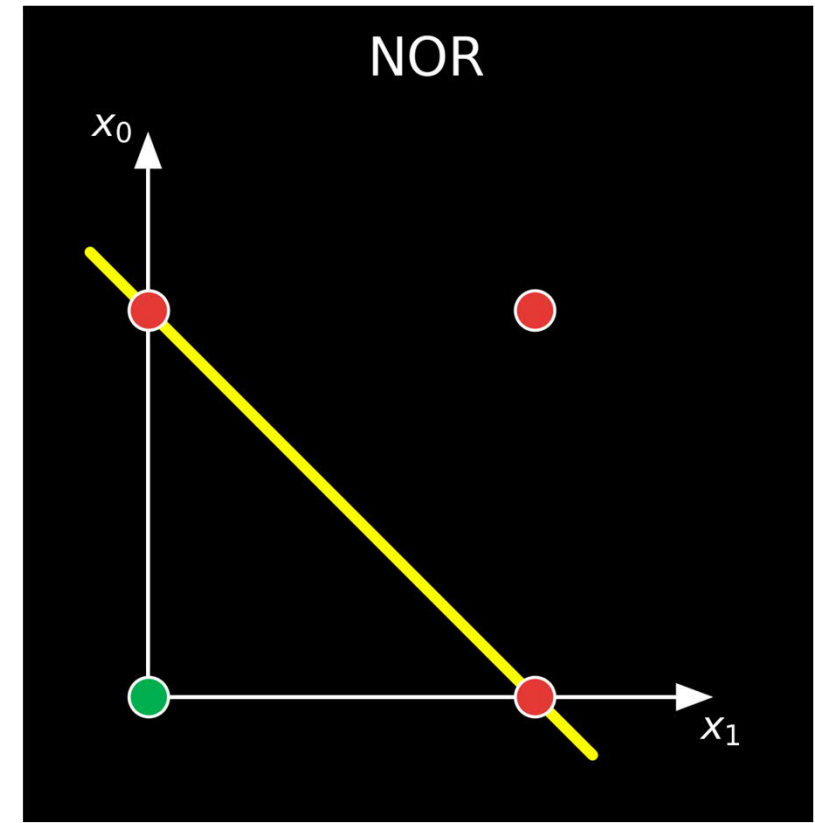
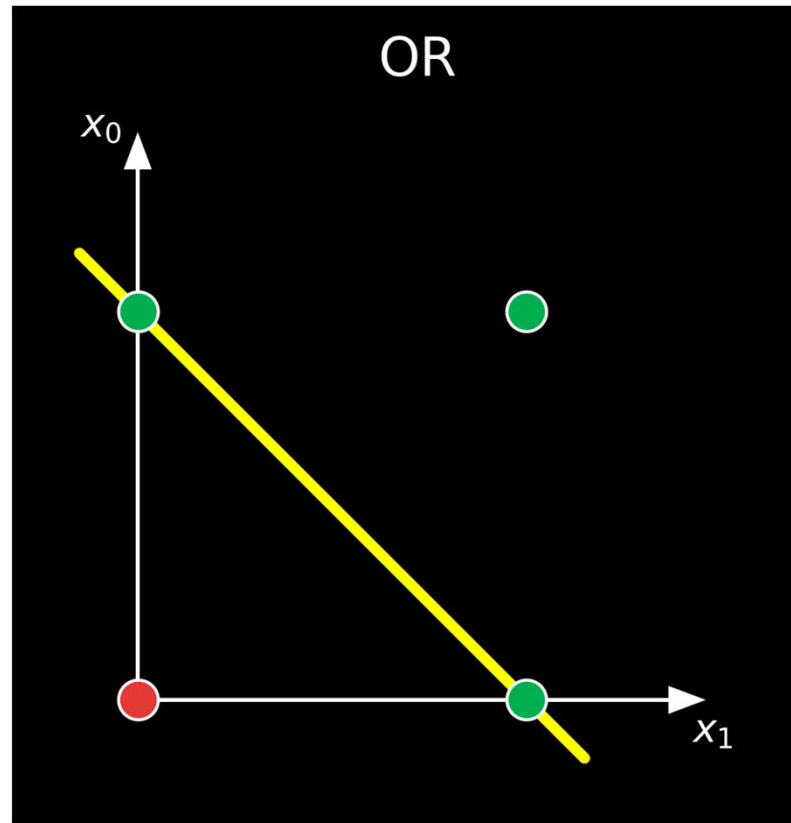
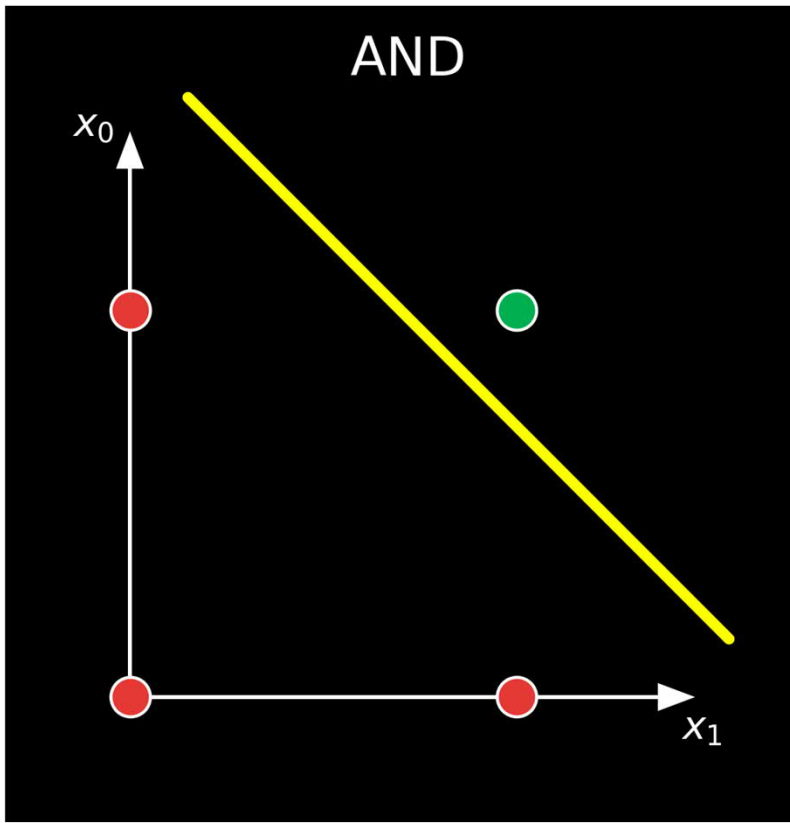
$$w_i \leftarrow w_i + \eta(y_{true} - y_{pred})x_i$$

oraz bias:

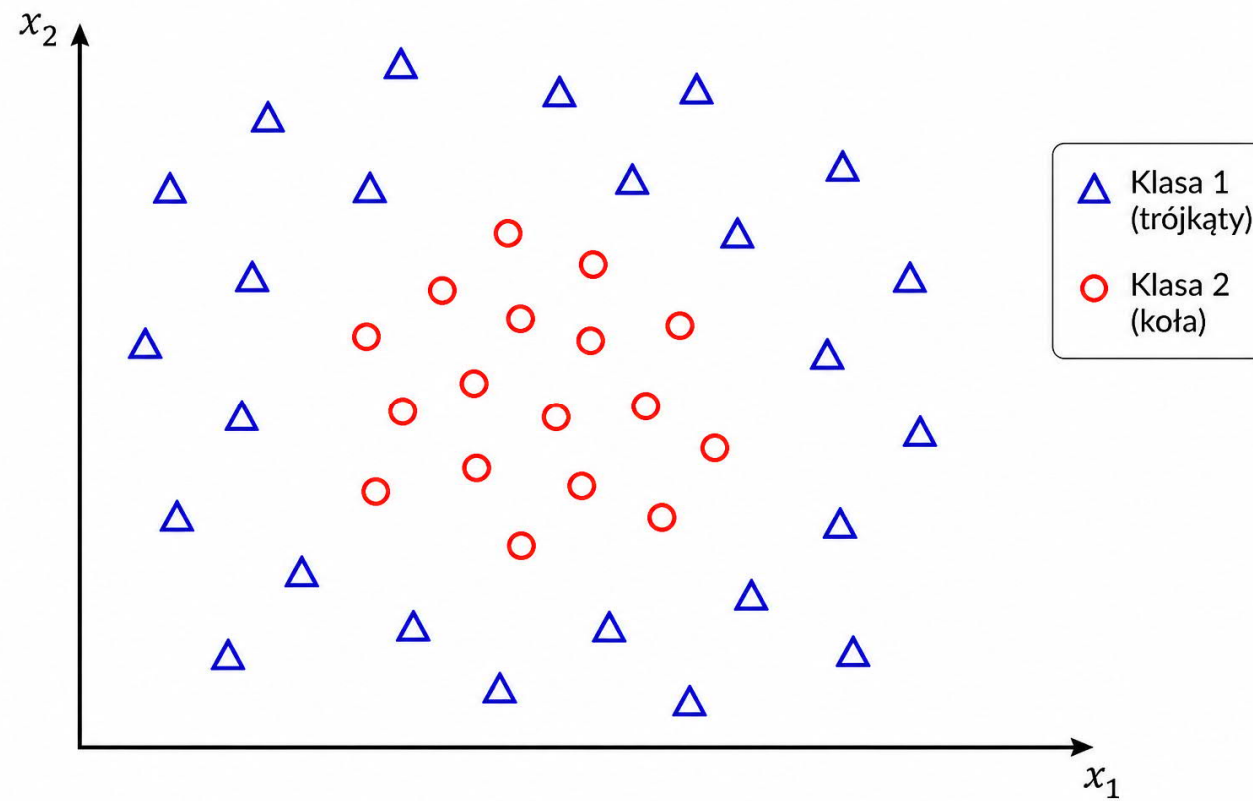
$$b \leftarrow b + \eta(y_{true} - y_{pred})$$

gdzie: η – współczynnik uczenia (np. 0.1)

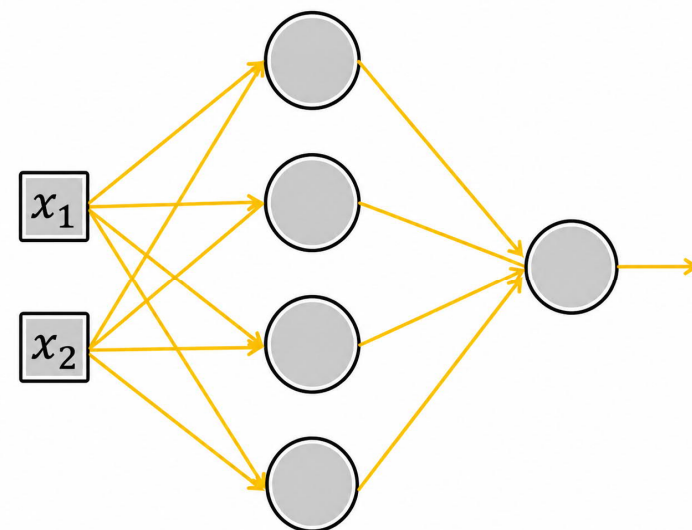
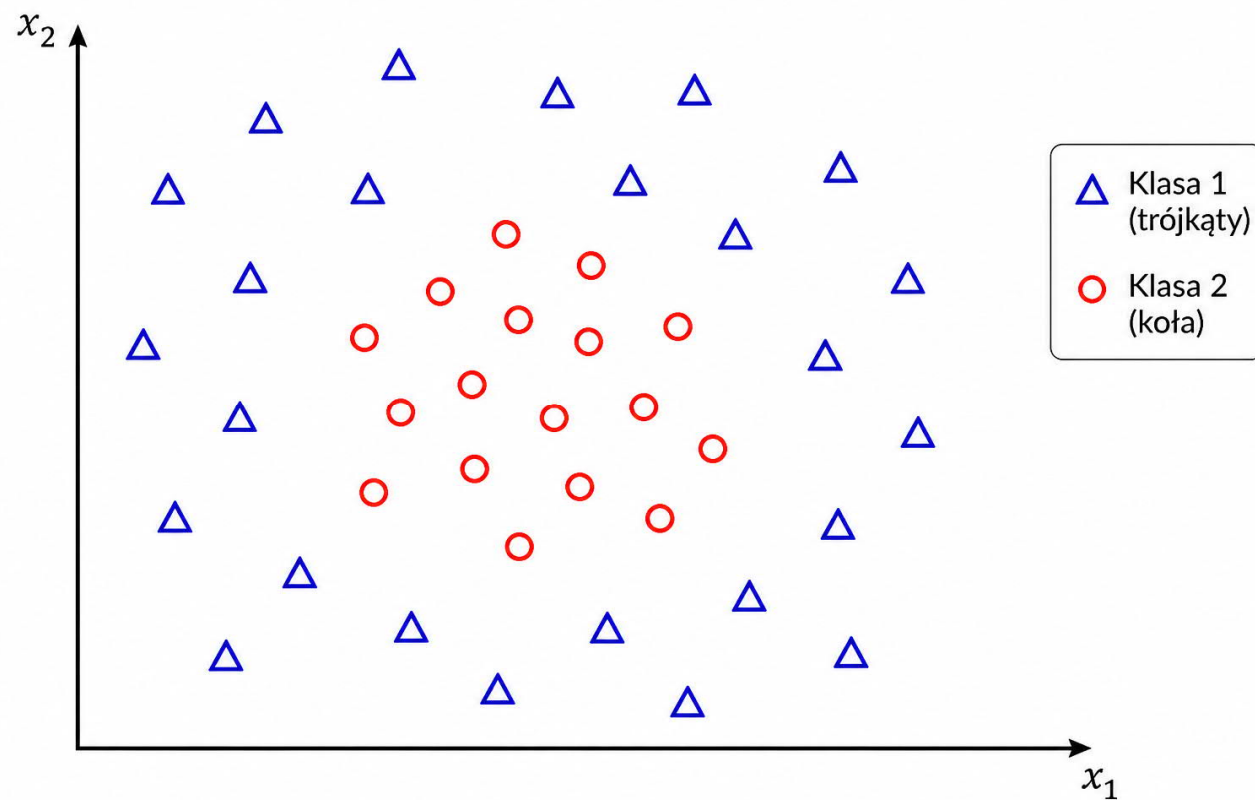
JAK WYGLĄDA UCZENIE W PERCEPTRONIE



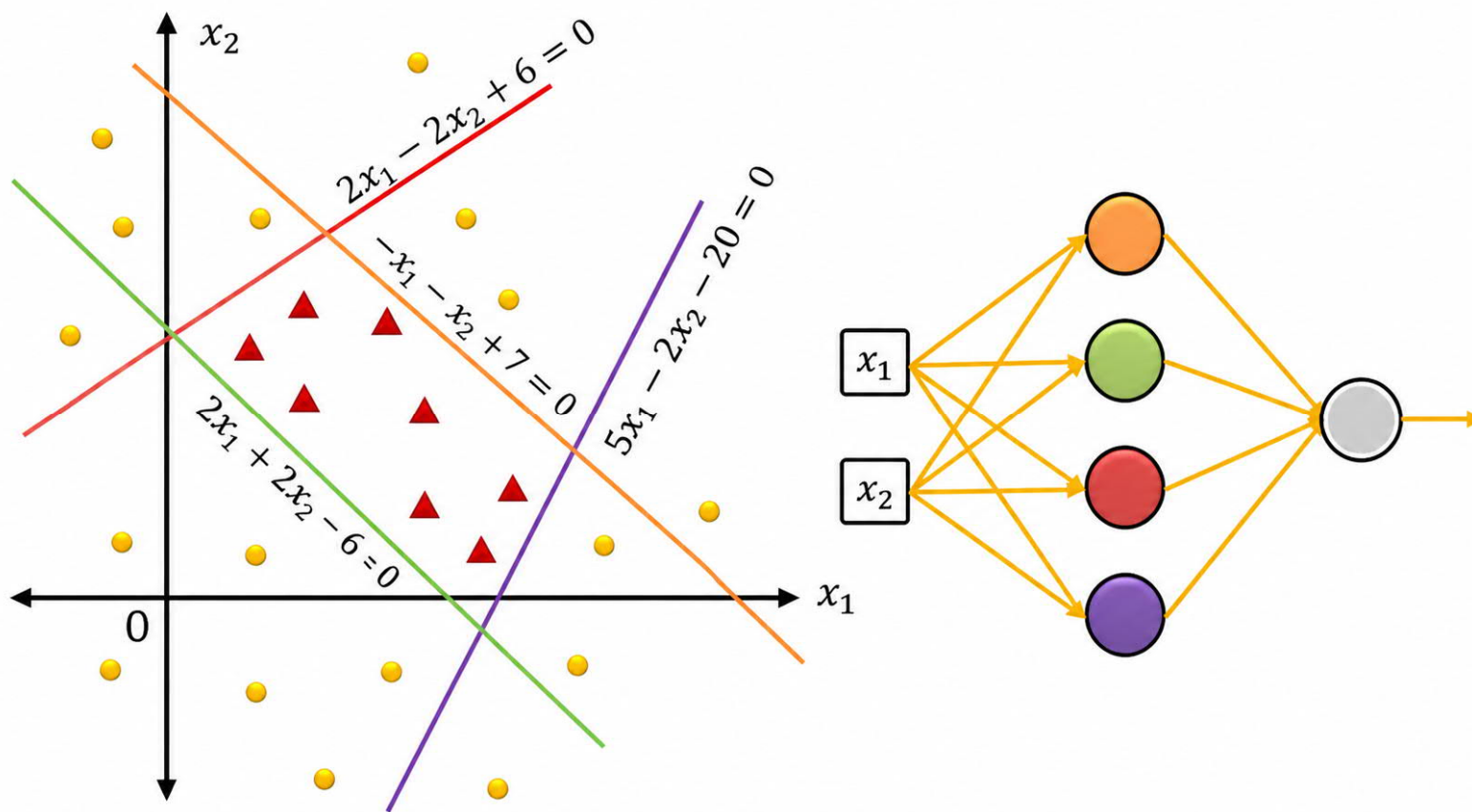
REALIZACJA BAMEK LOGICZNYCH W PERCEPTRONIE



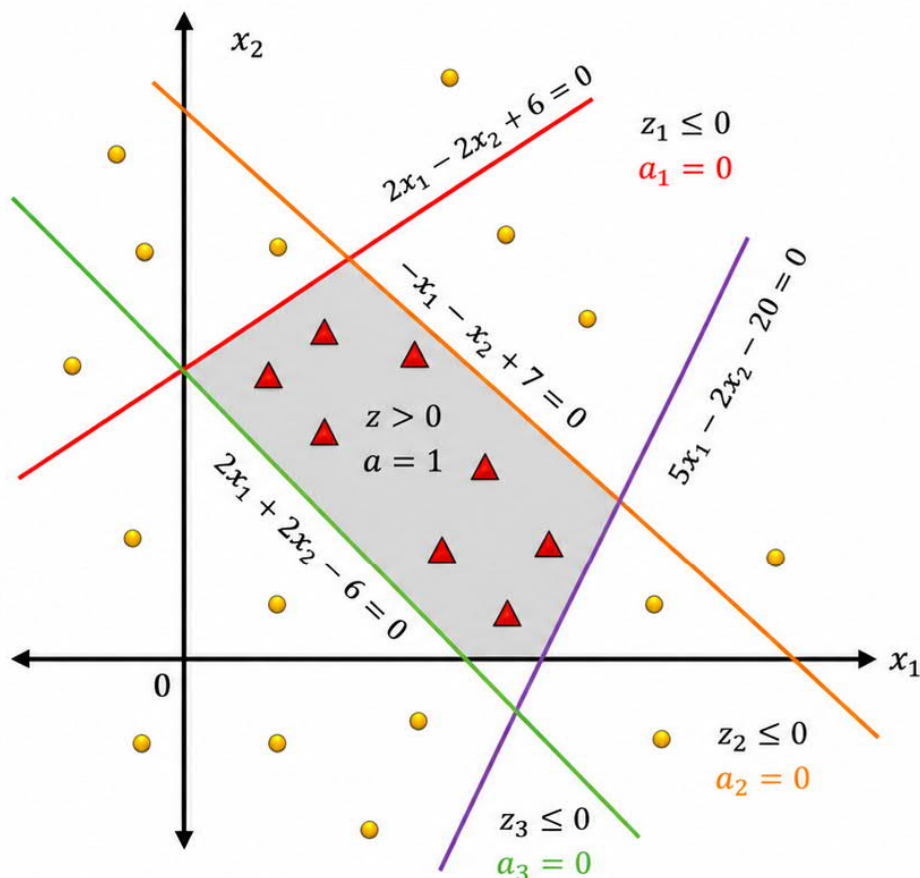
KLASYFIKACJA LINIOWA ZŁOŻONYCH REGIONÓW



KLASYFIKACJA LINIOWA ZŁOŻONYCH REGIONÓW



KLASYFIKACJA LINIOWA ZŁOŻONYCH REGIONÓW



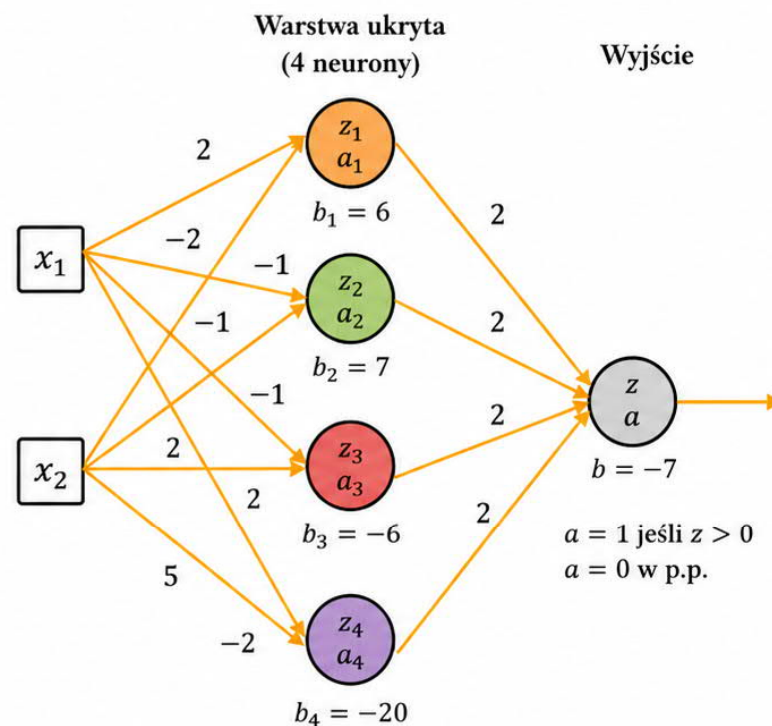
Każdy neuron ukryty realizuje:

Neuron 1:
 $z_1 = 2x_1 - 2x_2 + 6$
 $a_1 = \begin{cases} 1 & \text{jeśli } z_1 > 0 \\ 0 & \text{w p.p.} \end{cases}$

Neuron 2:
 $z_2 = -x_1 - x_2 + 7$
 $a_2 = \begin{cases} 1 & \text{jeśli } z_2 > 0 \\ 0 & \text{w p.p.} \end{cases}$

Neuron 3:
 $z_3 = 2x_1 + 2x_2 - 6$
 $a_3 = \begin{cases} 1 & \text{jeśli } z_3 > 0 \\ 0 & \text{w p.p.} \end{cases}$

Neuron 4:
 $z_4 = 5x_1 - 2x_2 - 20$
 $a_4 = \begin{cases} 1 & \text{jeśli } z_4 > 0 \\ 0 & \text{w p.p.} \end{cases}$



Neuron wyjściowy:

$$z = 2a_1 + 2a_2 + 2a_3 + 2a_4 - 7$$

$$a = \begin{cases} 1 & \text{jeśli } z > 0 \\ 0 & \text{w p.p.} \end{cases}$$

Każdy neuron ukryty tworzy jedną prostą.

Warstwa wyjściowa łączy je wagami i sprawdza czy punkt leży w wyznaczonym obszarze.

Wagi „2” wzmacniają wpływ neuronów ukrytych, a bias „-7” sprawia, że wszystkie muszą być aktywne.

TensorFlow Playground to interaktywne środowisko webowe, które pozwala wizualnie eksperymentować z działaniem sieci neuronowych bez konieczności pisania kodu.

Użytkownik może zmieniać parametry modelu, takie jak liczba warstw, liczba neuronów, funkcje aktywacji czy współczynnik uczenia, i natychmiast obserwować wpływ tych zmian na proces uczenia oraz granicę decyzyjną. Narzędzie umożliwia także wybór różnych zbiorów danych oraz cech wejściowych, co pomaga zrozumieć, kiedy model liniowy jest wystarczający, a kiedy potrzebne są bardziej złożone struktury.

Dzięki swojej wizualnej formie Playground jest szczególnie przydatny w edukacji, ponieważ w przystępny sposób pokazuje mechanizmy działania perceptronu i sieci wielowarstwowych.

TENSORFLOW PLAYGROUND



TensorFlow



Epoch
001,849

Learning rate
0.03

Activation
Linear

Regularization
None

Regularization rate
0

Problem type
Classification



TensorFlow

DATA

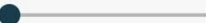
Which dataset do you want to use?



Ratio of training to test data: 30%



Noise: 0



Batch size: 10



REGENERATE

FEATURES

Which properties do you want to feed in?



X_1^2

X_2^2

X_1X_2

$\sin(X_1)$

$\sin(X_2)$

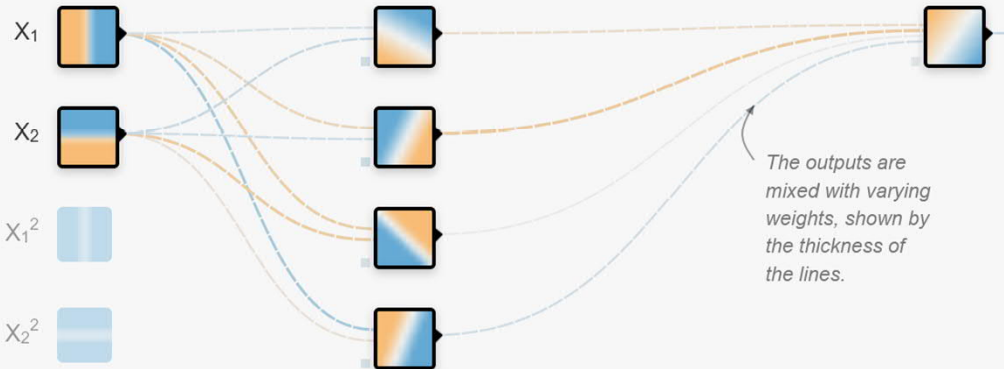
+ - 2 HIDDEN LAYERS



4 neurons



1 neuron

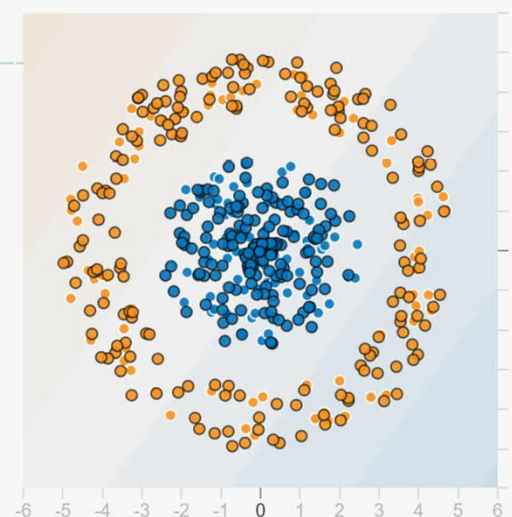
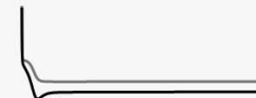


The outputs are mixed with varying weights, shown by the thickness of the lines.

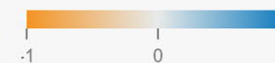
This is the output from one neuron. Hover to see it larger.

OUTPUT

Test loss 0.497
Training loss 0.498



Colors shows data, neuron and weight values.



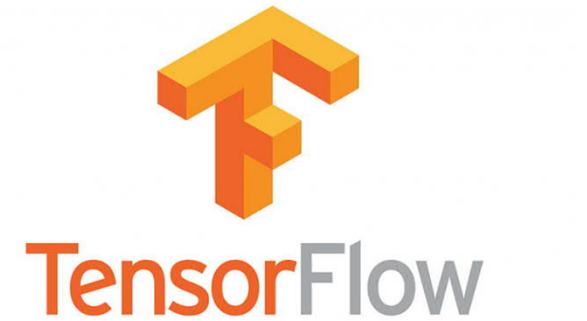
☒ Show test data ☐ Discretize output

1. Problem (Problem Type)

Classification (klasyfikacja)
przypisywanie punktów do klas (np. 0 / 1)

Regression (regresja)
przewidywanie wartości ciągłych

Decyduje o tym, jak interpretowany jest wynik sieci



TENSORFLOW PLAYGROUND

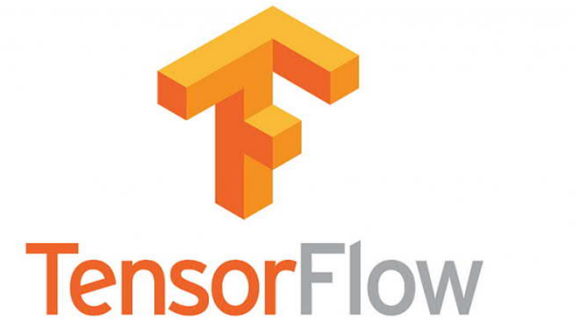
2. Dataset (zbiór danych)

Możesz wybrać różne typy danych:

- Circle – dane nieliniowe (okręgi),
- XOR – klasyczny problem nieliniowy,
- Gaussian – skupiska punktów,
- Spiral – bardzo trudny problem.

Pozwala testować:

- kiedy perceptron działa,
- kiedy potrzebna jest głębsza sieć.



TENSORFLOW PLAYGROUND

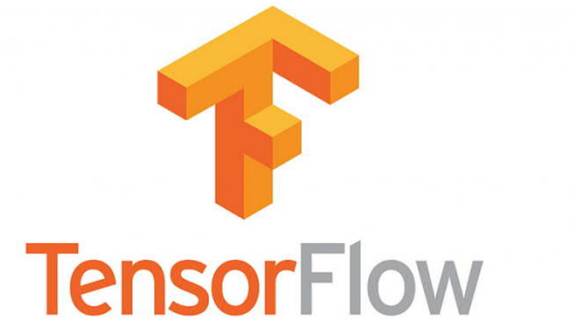
3. Noise (szum)
dodaje losowość do danych.

Im większy:
tym trudniejsze zadanie,
większe ryzyko przeuczenia.

4. Features (cechy wejściowe)

Możesz wybrać: x_1, x_2 ; x_1^2, x_2^2 ; $x_1 \cdot x_2$; $\sin(x), \cos(x)$

dodanie cech = zwiększenie możliwości modelu
nawet prosty model może rozwiązać trudny problem



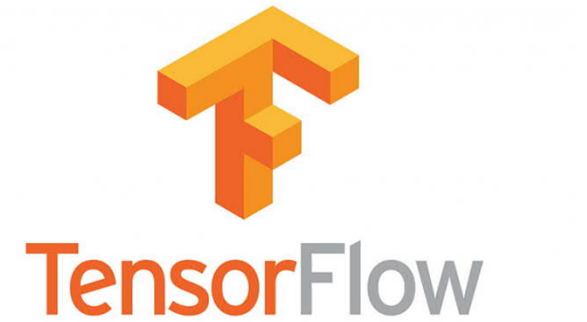
TENSORFLOW PLAYGROUND

5. Network Shape (architektura sieci)

- liczba warstw ukrytych,
- liczba neuronów w każdej warstwie.

Wpływa na:

- zdolność modelu do nauki,
- złożoność granicy decyzyjnej.

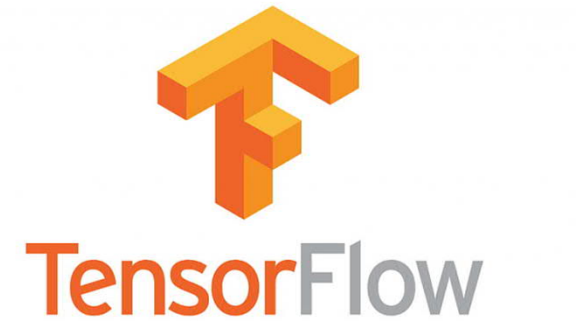


TENSORFLOW PLAYGROUND

6. Activation Function (funkcja aktywacji)

Dostępne opcje:

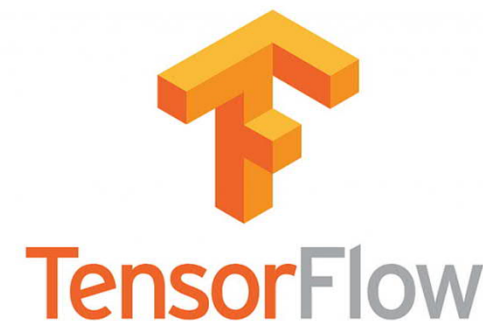
- Linear → brak nieliniowości,
- ReLU → szybka i popularna,
- Tanh → wartości od -1 do 1,
- Sigmoid → wartości od 0 do 1.



TENSORFLOW PLAYGROUND

7. Learning Rate (współczynnik uczenia)
określa szybkość uczenia

- zbyt duży: niestabilność
- zbyt mały: wolne uczenie



TENSORFLOW PLAYGROUND

8. Batch Size

liczba próbek w jednej iteracji

Wpływa na:

- Stabilność uczenia,
- Szybkość uczenia,

9. Regularization (regularyzacja)

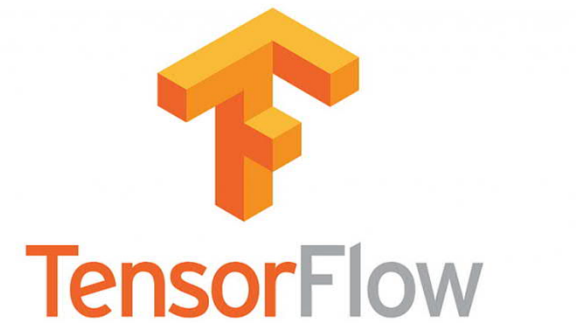
zapobiega przeuczeniu

Typy:

- L1
- L2

zmniejsza złożoność modelu

TENSORFLOW PLAYGROUND



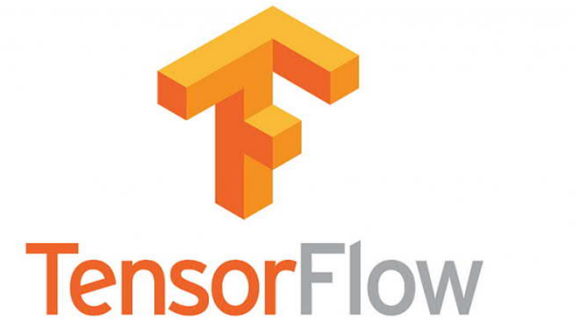
10. Regularization Rate siła regularyzacji

Większa wartość:

- prostszy model,
- mniej overfittingu.

11. Training Data (%) procent danych treningowych

reszta = dane testowe



TENSORFLOW PLAYGROUND

12. Show Test Data
pokazuje dane testowe

Pozwala ocenić czy model generalizuje.

13. Discretize Output
upraszcza wizualizację wyników

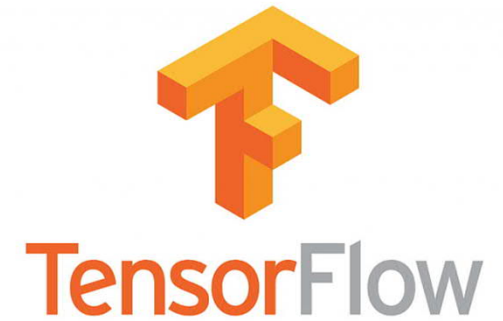
Zamiast gradientu prowadzi do wyraźniejszych klas.

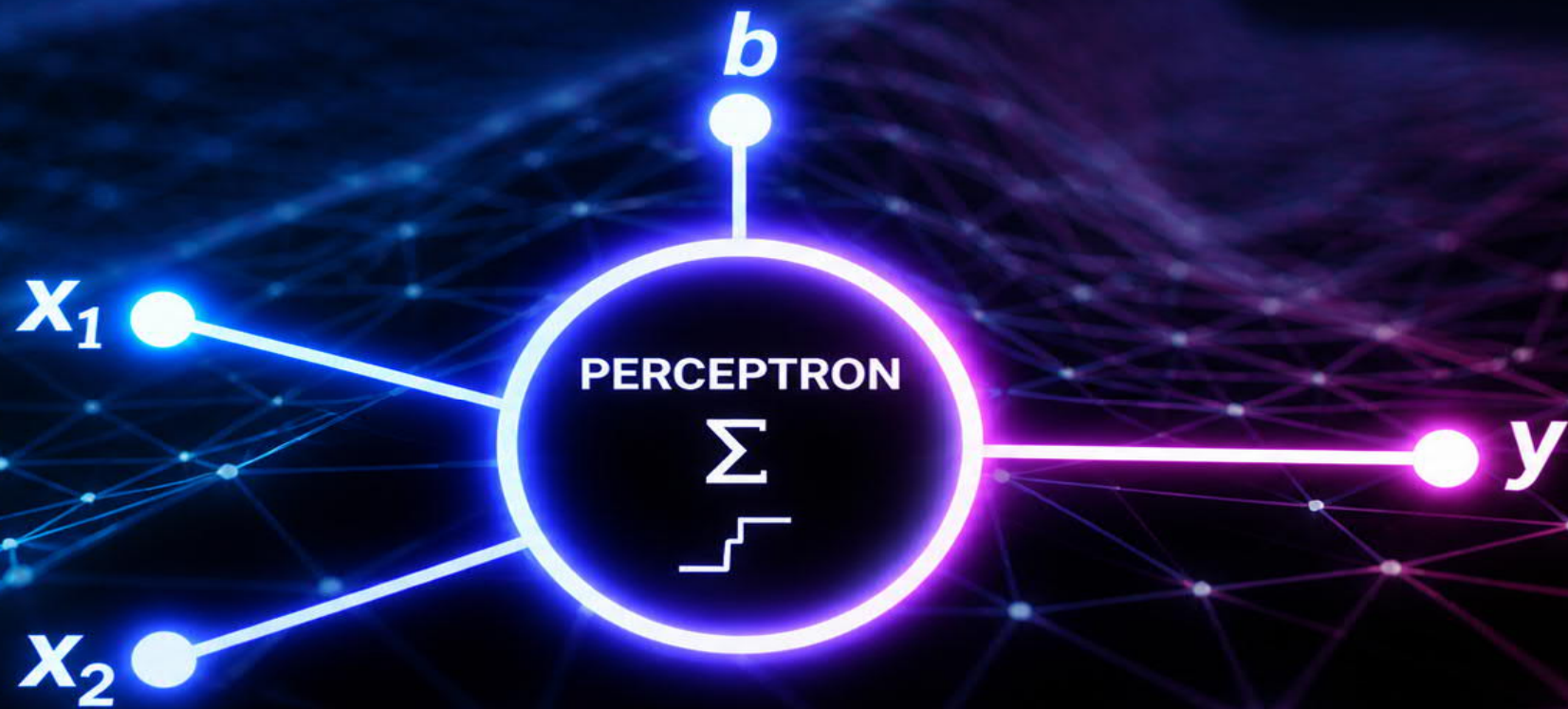
14. Seed
ustala losowość

Ten sam seed to te same wyniki.

TENSORFLOW PLAYGROUND

15. Collect Stats
zbiera statystyki uczenia przydatne do analizy





DZIĘKUJĘ ZA UWAGĘ